

# Object-Oriented Programming (OOP)-CS101

## Introduction to OOP

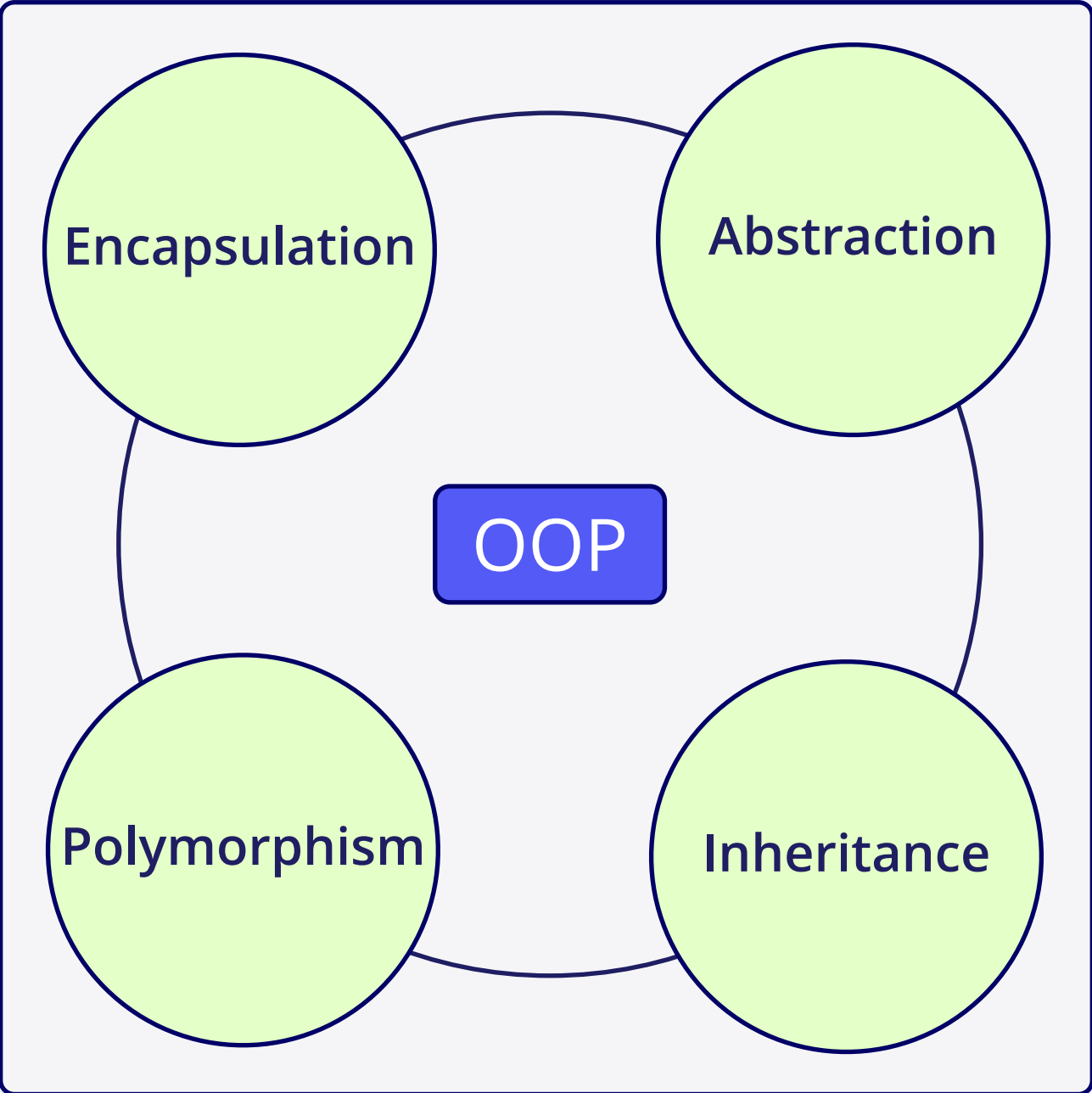
Object-oriented programming (OOP) is a programming paradigm that organizes code using objects, which are instances of classes.

## Benefits of OOP



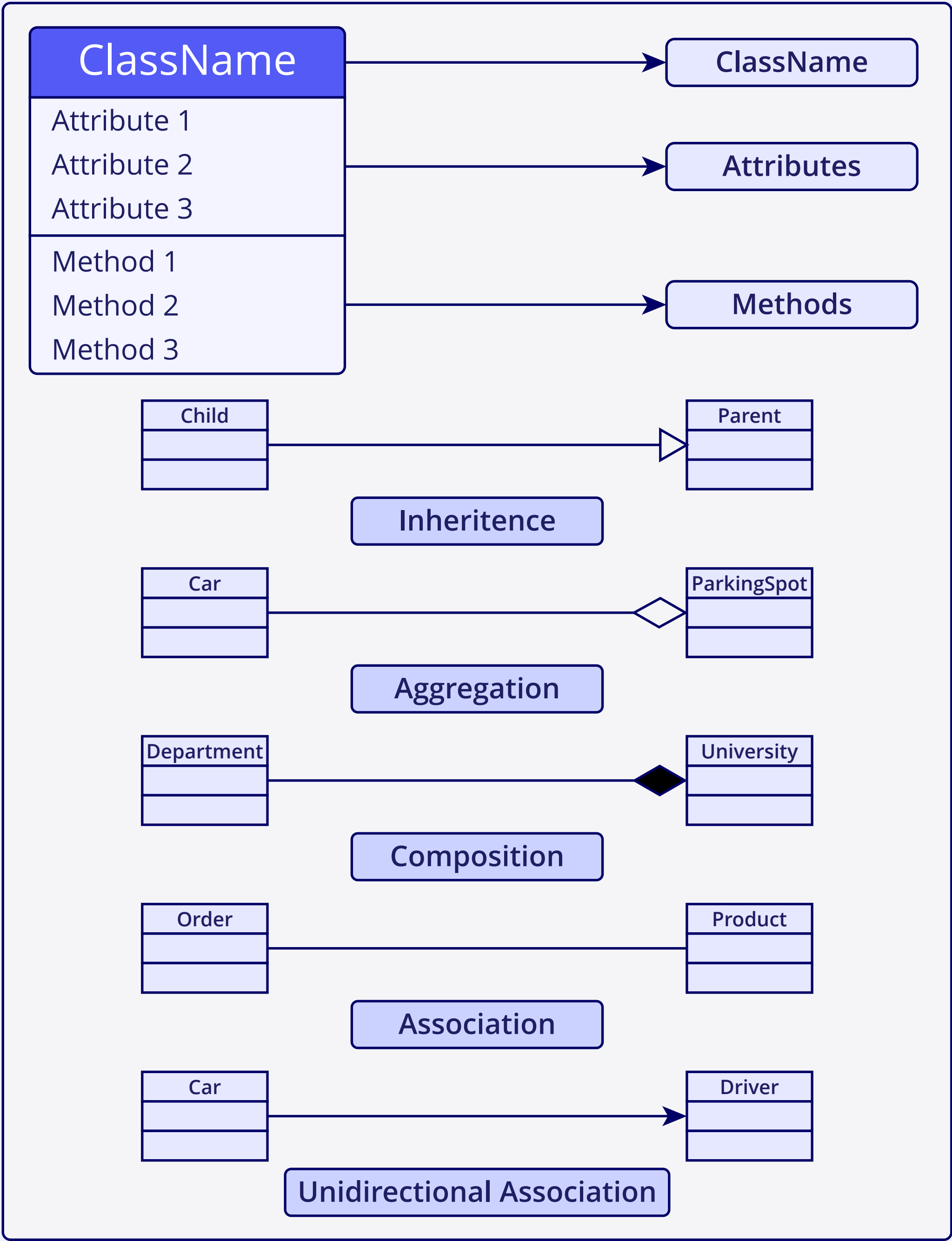
## Principles of OOP

The following four principles form the foundation of OOP and contribute to the creation of modular, scalable, and maintainable software systems.



# Class Diagram

A class diagram is a visual representation in UML depicting the structure and relationships among classes in a software system, showcasing attributes, methods, and associations.





# Classes and Objects

## Class

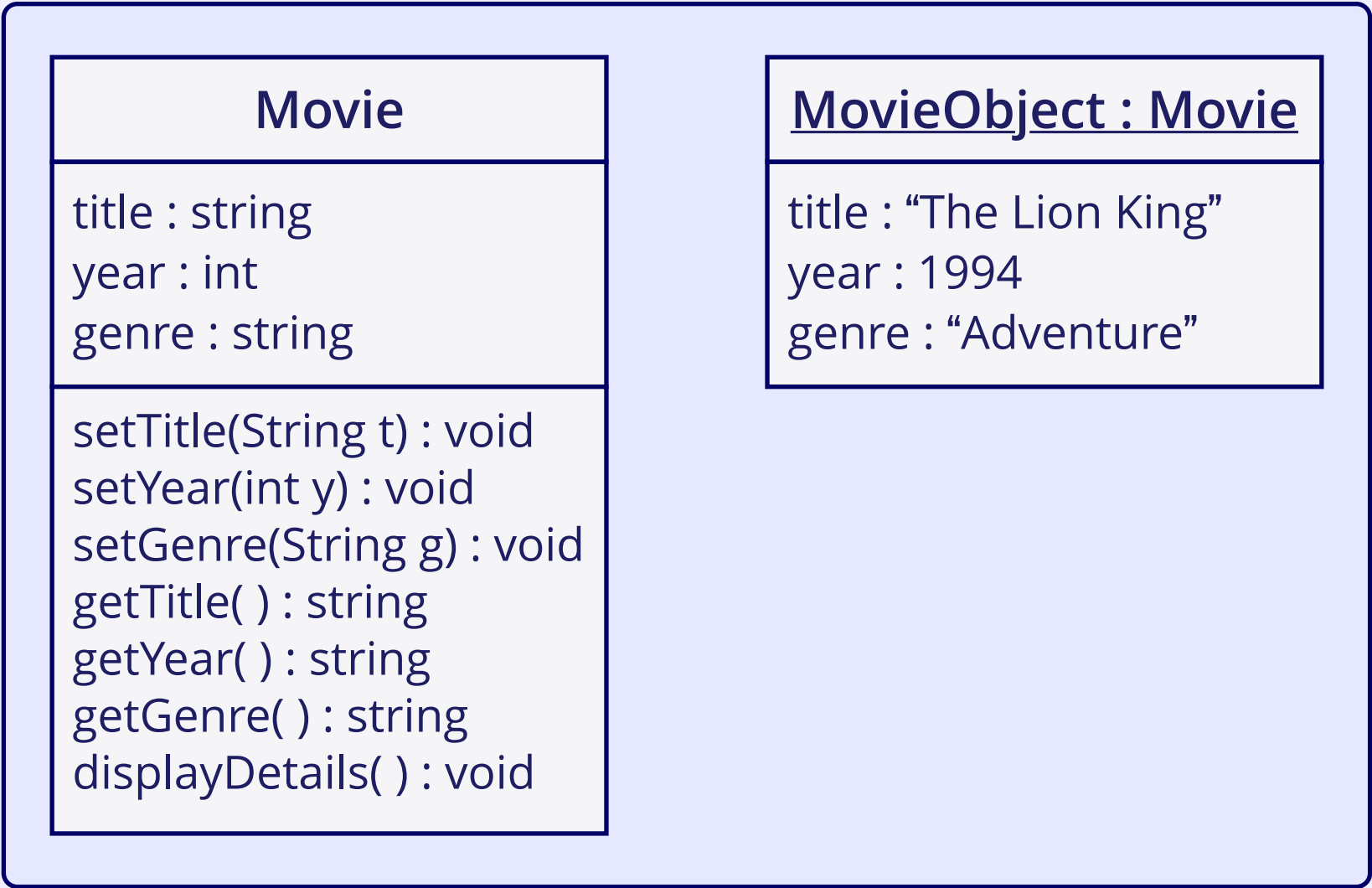
A class is a blueprint for creating objects. For example, a **Person** class may have attributes like **name** and methods like **eat()**.

## Object

An object is an instance of a class. It represents a real-world entity. For instance, an object of the **Person** class could be **John**.

## Class vs. Object

A class is a template, and an object is an instance of that template.



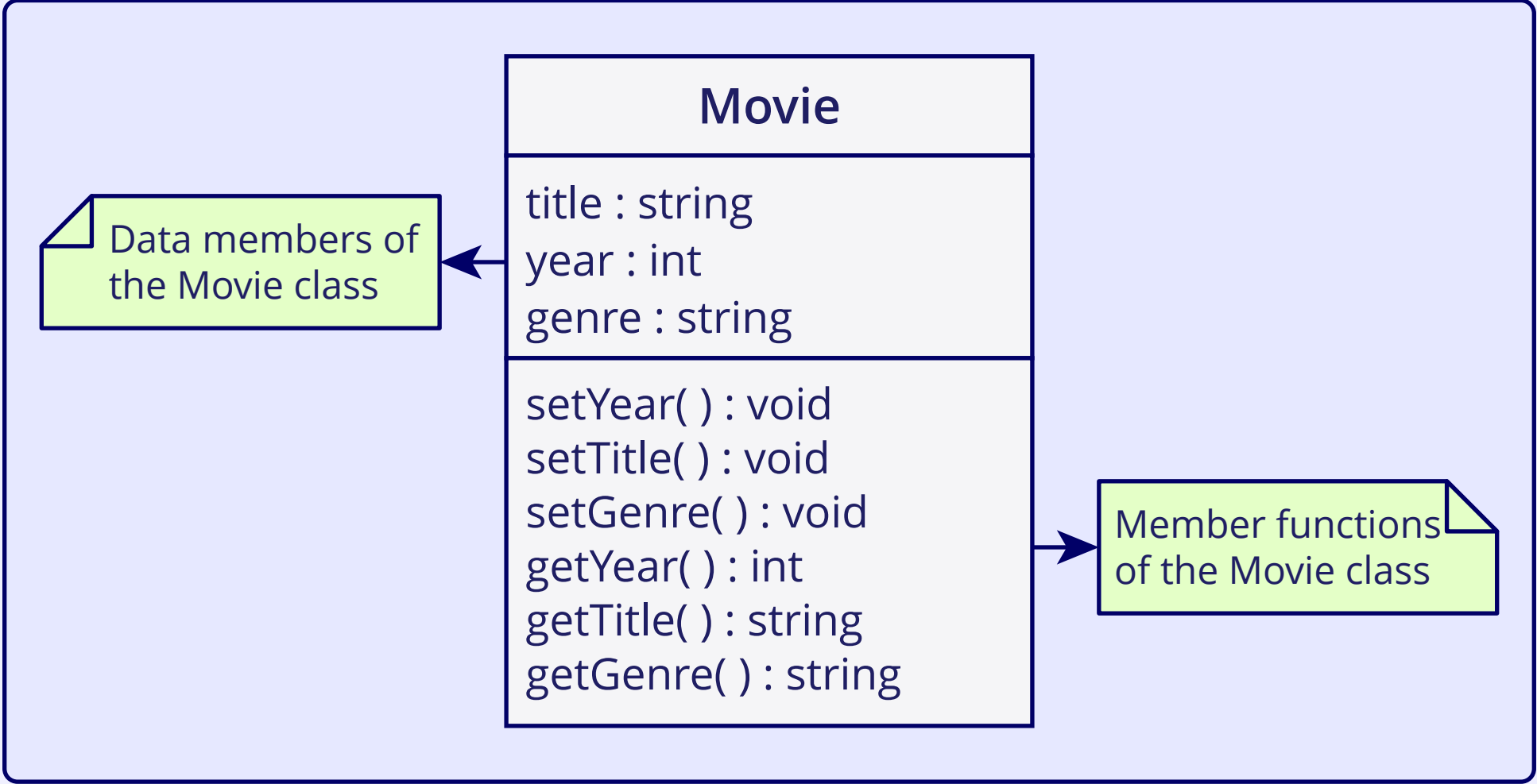
# Attributes and Methods

## Attributes (Properties, Fields, Members)

Attributes are variables that store properties of the object of the class.

## Methods (Functions, Procedures)

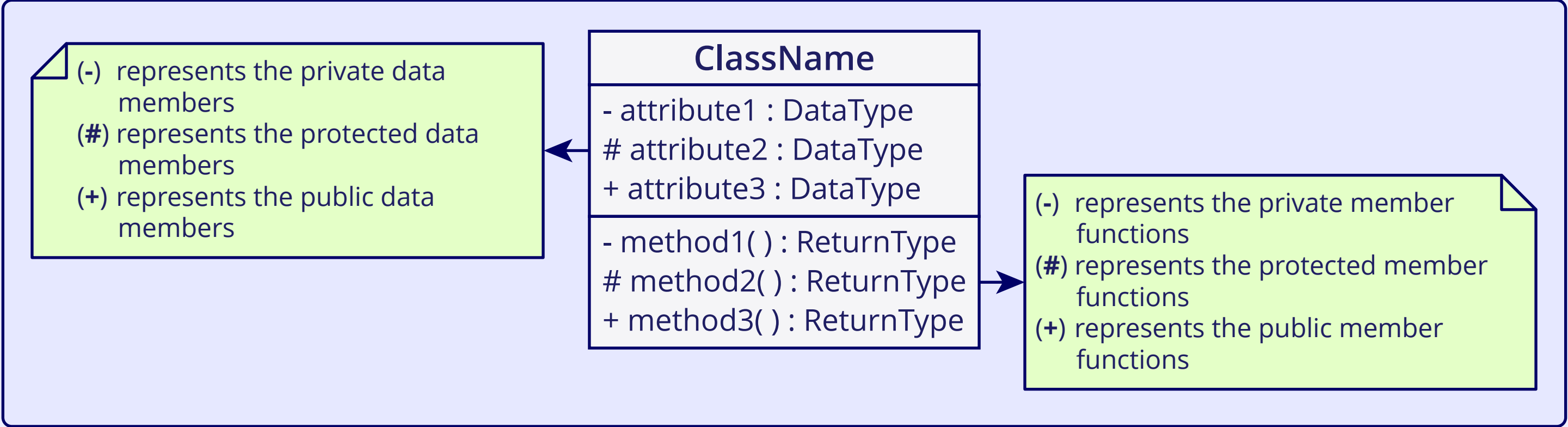
Methods are functions that perform actions on objects.



# Access Modifiers

Access modifiers control the visibility of class members:

- **Public:** Members are accessible from any part of the program.
- **Private:** Members are accessible only within the class.
- **Protected:** Members are accessible within the class and its derived classes.



# Encapsulation

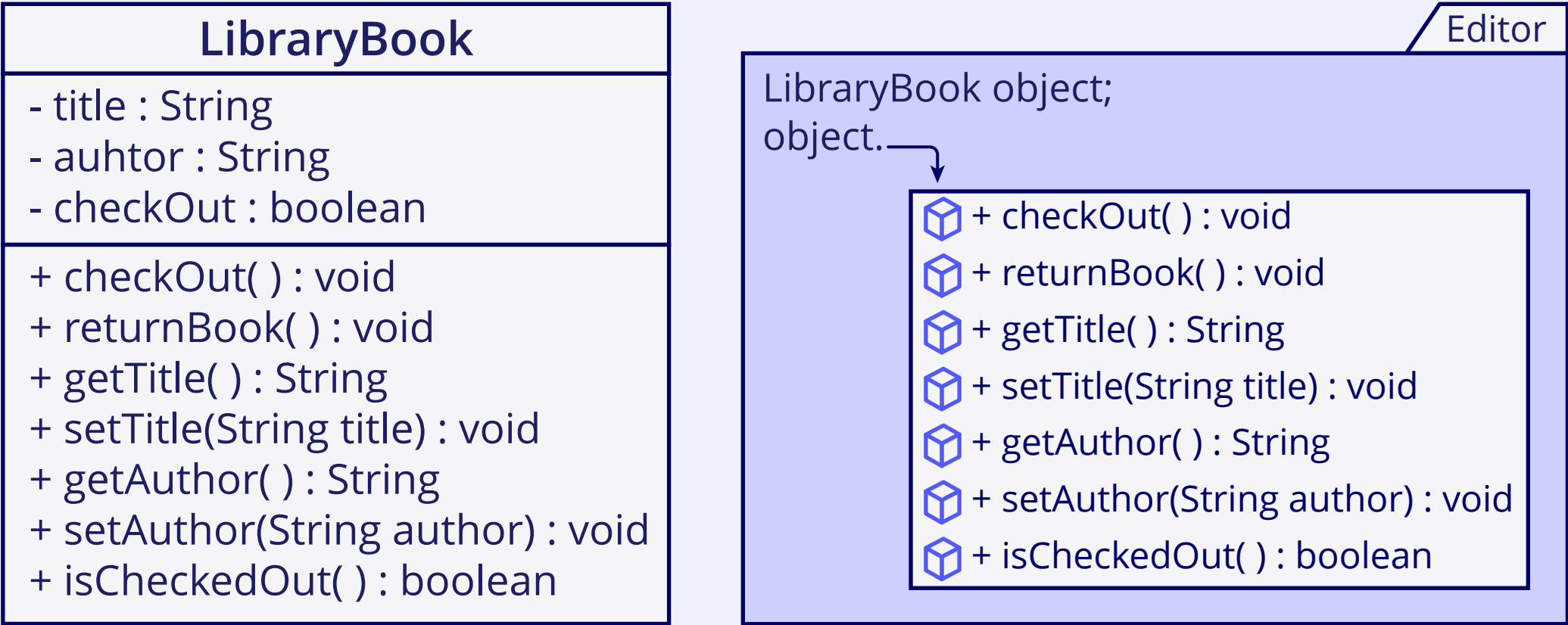
Encapsulation refers to bundling data and methods that operate on that data within a single unit (class).

## Use of Access Modifiers

Access modifiers control visibility, ensuring data integrity and security.

## Getters and Setters

Getters retrieve attribute values, and setters modify them.



# Association

Association is a mechanism to provide communication or services between one object and another.

## Class Association (Inheritance)

Inheritance allows a class (derived/child class) to inherit attributes and methods from another class (base/parent class).

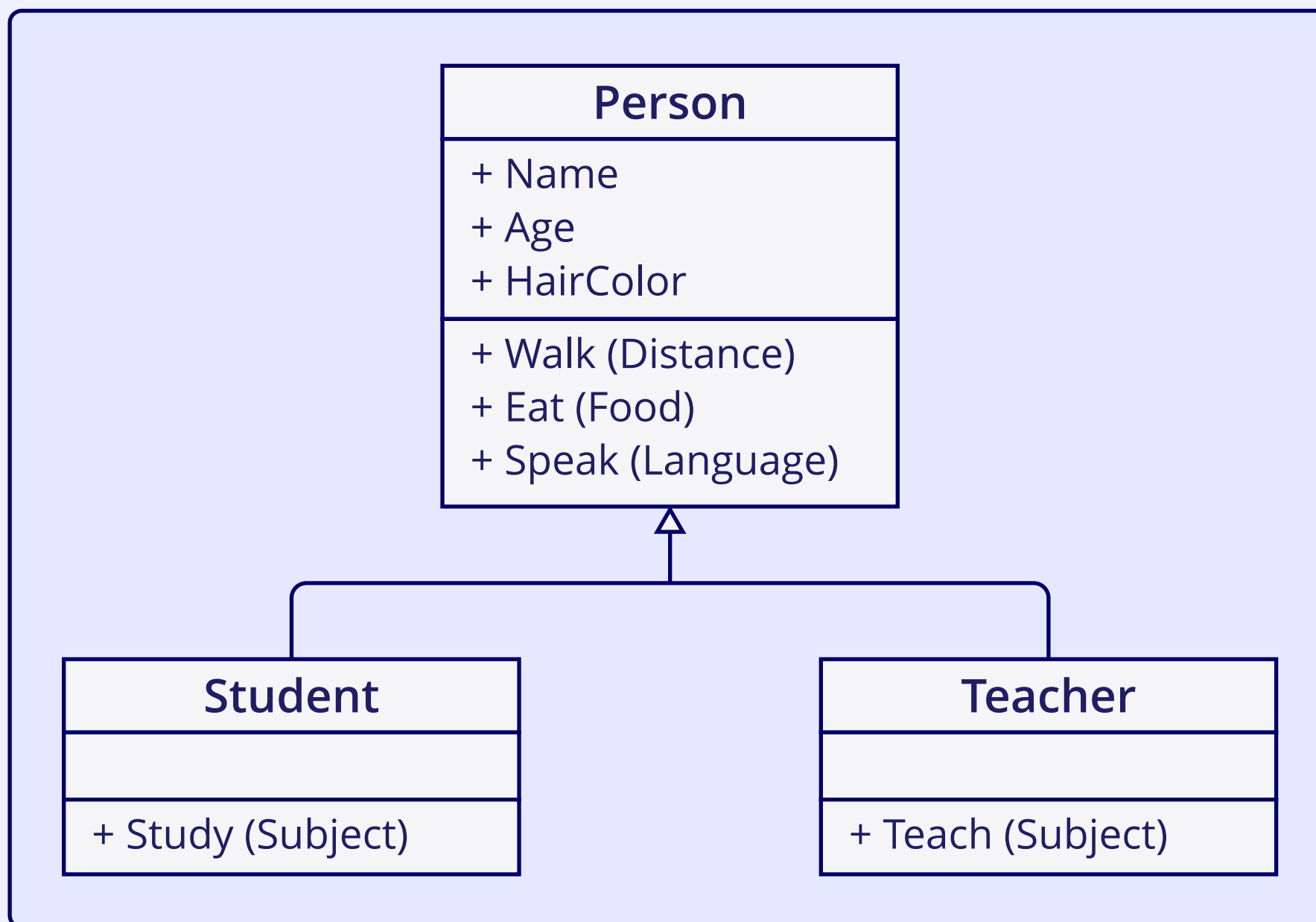
### Base class (parent class, superclass)

The class whose properties and methods are inherited



## Derived class (child class, subclass)

The class that inherits properties and methods



The derived classes, **Student** and **Teacher**, inherit properties and methods from the **Person** class.

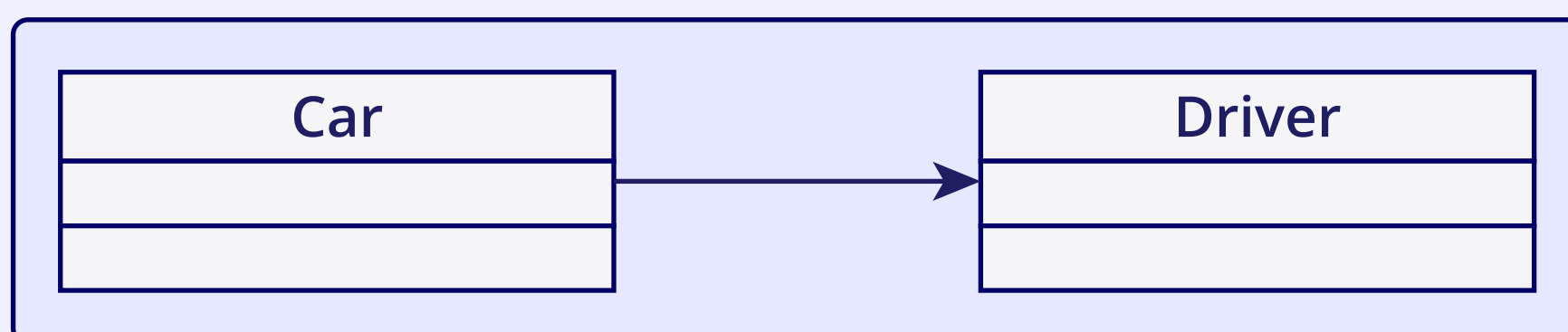
## Object Association

Object association (the relationship between objects) can be divided into the following categories:

- Simple association
- Aggregation
- Composition

## Simple association

Simple association is achieved through reference, which one object can inherit from another. It is the weakest connection between objects.



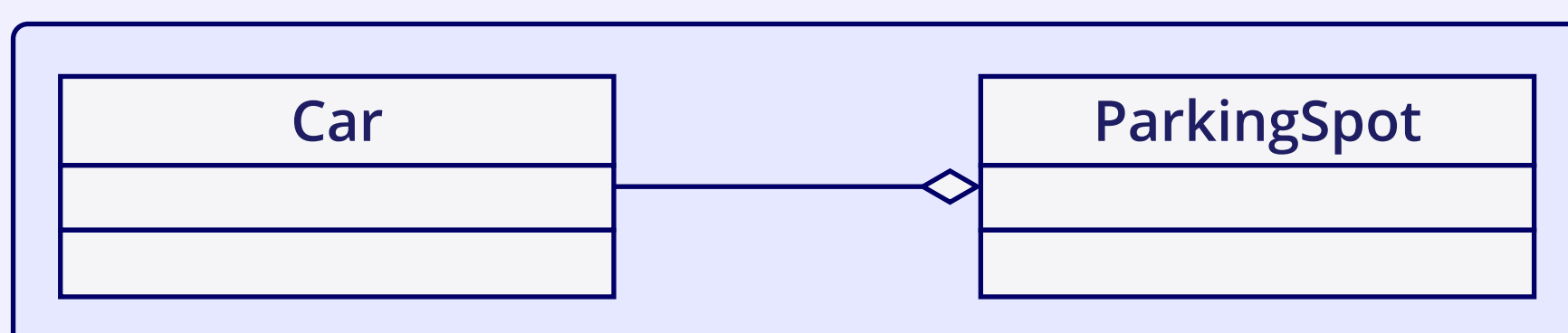
A car can be driven by one or more drivers, while the driver is not directly associated with a specific car instance.

## Aggregation

Aggregation describes the relationship between the container and the object it contains. An object may contain an aggregate of another object.

Aggregation is a weak relationship because:

- Aggregate objects are not a part of the container.
- Aggregate objects can exist independently.



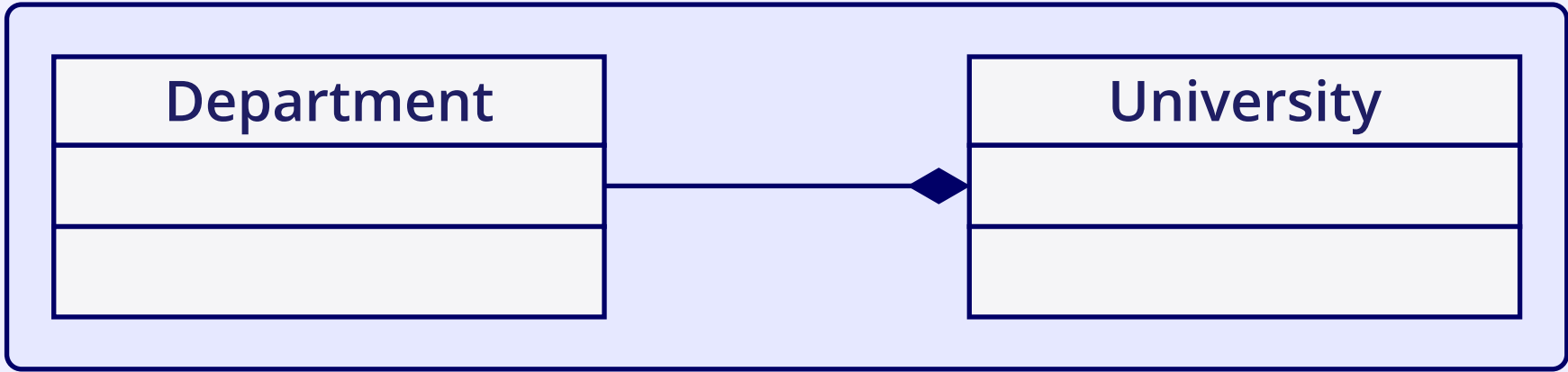
The **Car** (part) class is not intrinsically tied to the **ParkingSpot** (whole) class and can exist independently.

## Composition

Composition describes the relationship between smaller “part” objects and larger “whole” objects.

Composition is a strong relationship because:

- Composed objects become a part of the composer.
- Composed objects can not exist independently.



The **Department** (part) class is closely tied to and dependent on the **University** (whole) class and can’t exist independently.

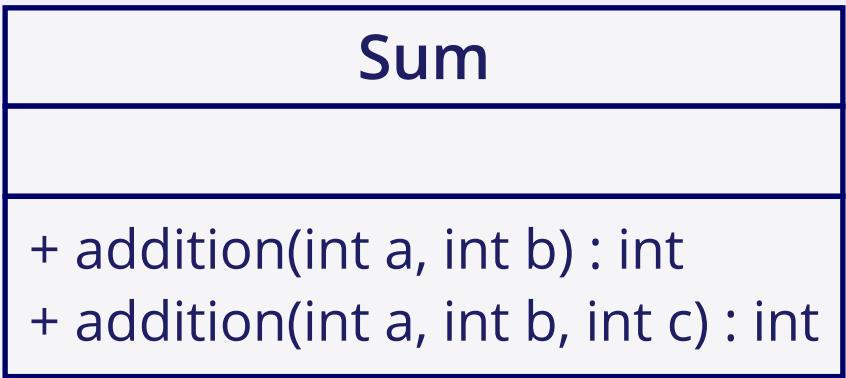
## Polymorphism

Polymorphism allows objects of different types to be treated as objects of a common type.

### Types of Polymorphism

#### Compile-time polymorphism (method overloading)

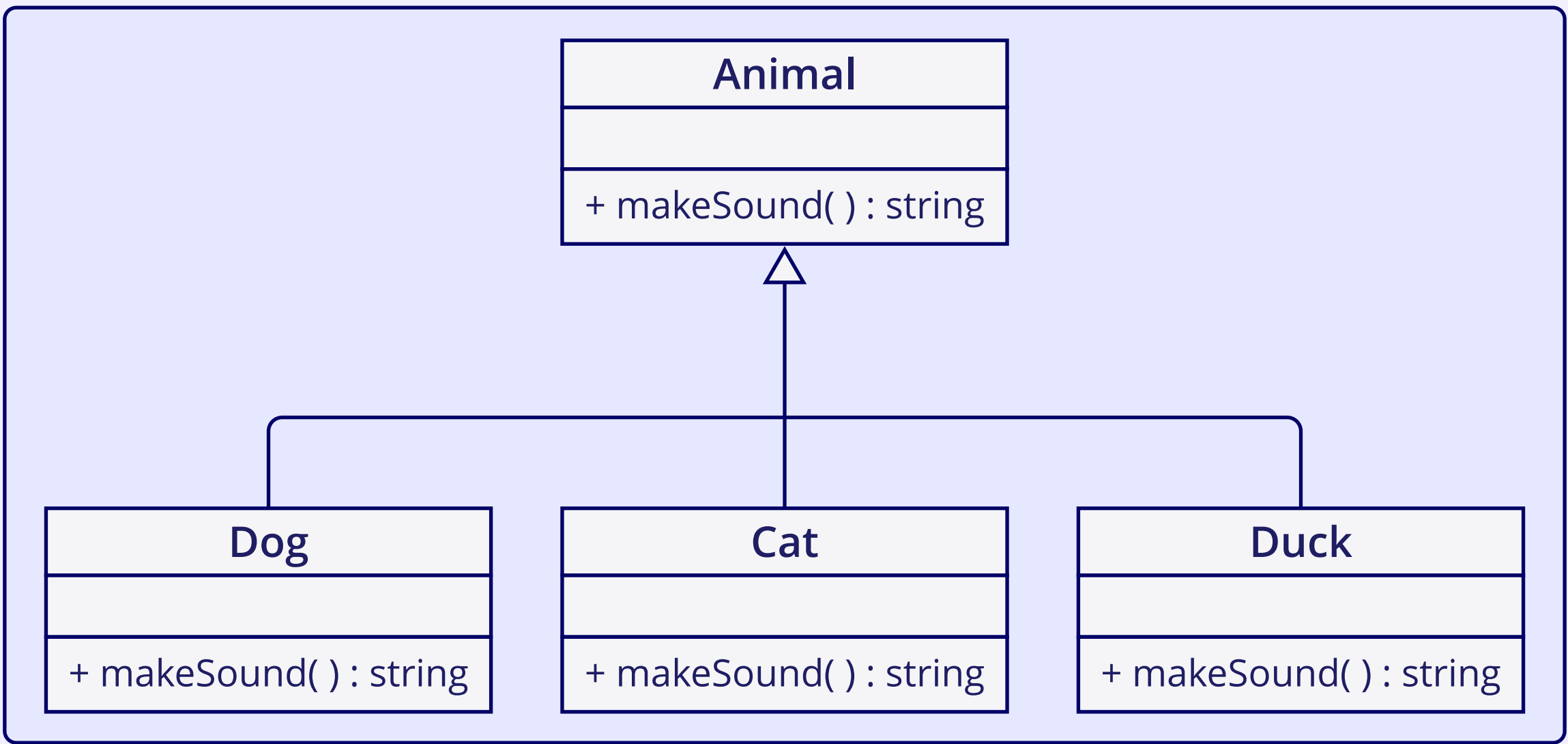
Methods with the same name but different parameters



An object of the **Sum** class is instantiated within a specific context. Any of these overloaded functions can be called based on the provided parameters.

#### Runtime polymorphism (method overriding)

A method in a base class is redefined in the derived class.





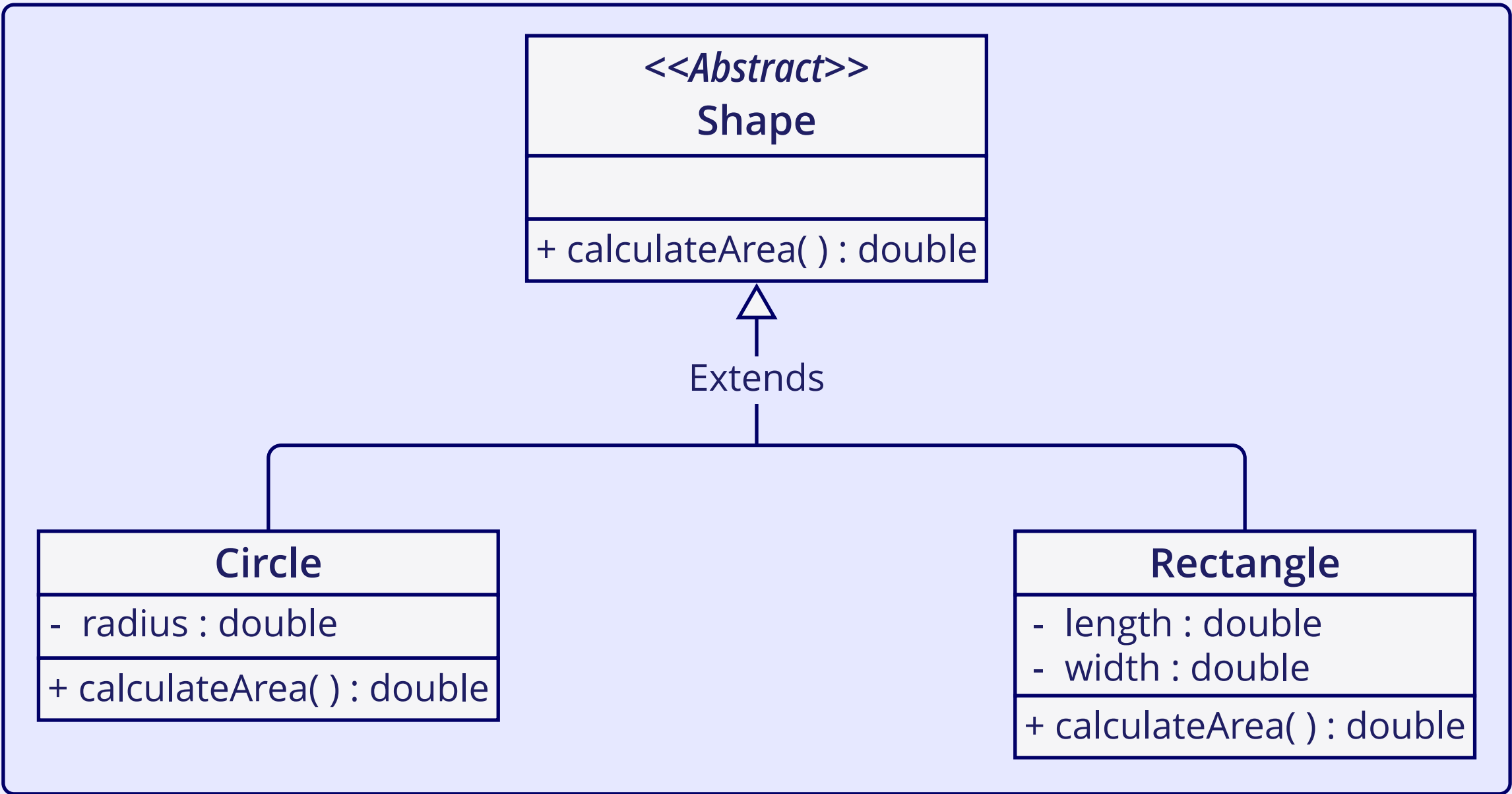
In this example, the **makeSound** method exhibits runtime polymorphism. When invoked through polymorphic reference, the overridden method in each derived class is dynamically executed, demonstrating the flexibility of the code at runtime.

# Abstraction

Abstraction focuses on essential features while hiding nonessential details.

## Abstract Classes

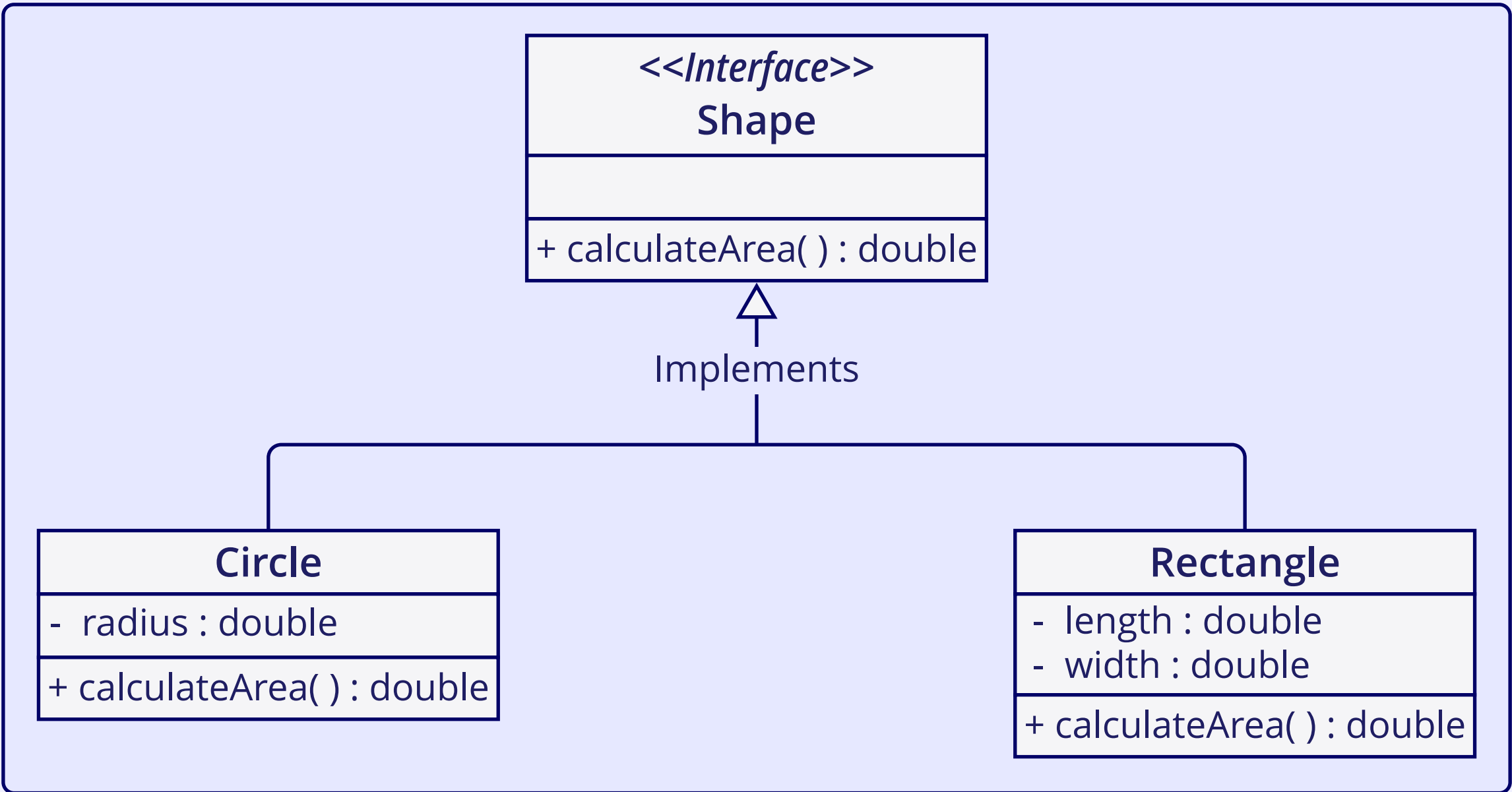
These classes cannot be instantiated and may have abstract methods.



- Utilizing the abstract class reference allows interaction with specific implementations without revealing the detailed internal workings.
- Abstraction can also be achieved using interfaces.

## Interfaces

They define a contract of method signatures for classes to implement, facilitating abstraction and achieving multiple inheritance.



- By creating objects using the interface reference, the code achieves abstraction, enabling interaction with diverse shapes without the need to understand their specific implementations.

- This design promotes flexibility and upholds a clear separation between the interface and the implementation, reinforcing the principles of abstraction in object-oriented programming.

### Abstract Classes vs. Interfaces

| Feature          | Abstract Classes                                    | Interfaces                                                     |
|------------------|-----------------------------------------------------|----------------------------------------------------------------|
| Instantiation    | Cannot be instantiated                              | Cannot be instantiated                                         |
| Inheritance      | Supports single inheritance                         | Supports multiple inheritance                                  |
| Constructors     | Can have constructors (with or without)             | Cannot have constructors                                       |
| Access modifiers | Can be public, protected, or private                | All methods are implicitly public                              |
| Fields/Variables | Can have fields (with any access level)             | Can only have constants (public, static, or final)             |
| Use cases        | Suitable for common code sharing and code extension | Suitable for defining common behavior across unrelated classes |

## Common OOP Languages

Popular languages that support the OOP paradigm include Java, C#, C++, and Python.

## Tips for Effective OOP Programming

1

Use meaningful names for classes and methods to enhance code readability.

2

Design classes with a single responsibility, promoting modularity.

3

Protect the internal state and control access to class members.

4

Hide unnecessary details, exposing only essential functionalities.

5

Provide clear and concise documentation for classes and methods.

6

Follow a consistent coding style for easier code maintainability.

7

Implement effective error-handling mechanisms to create robust code.

8

Stay up-to-date with new OOP concepts and best practices.